

# Deriving Scaling Laws from Scratch

Nikos Fourlis

April 2026

## Overview

This report presents an empirical derivation of scaling laws for a small Greek-language language model. Three models of varying sizes were each trained to 200M tokens, with evaluation checkpoints at 2M, 20M, and 200M tokens (9 runs total). The resulting validation losses were used to fit power-law relationships between model performance, parameter count, training tokens, and compute (FLOPs).

---

## Dataset

**Source:** alexliap/tinystories-gr — 2.14M Greek children’s stories translated from the TinyStories corpus.

### Preprocessing:

- Only the greek\_translation column was used; English text was never seen by the model
  - Text was tokenized and concatenated with `<|endoftext|>` separators
  - Flattened token stream was chunked into sequences of length 257 (256 input + 1 target)
  - Chunks were shuffled once with a fixed seed (`numpy.random.default_rng(42)`) and cached to disk
  - 90/10 train/val split applied after shuffling
- 

## Tokenizer

A BPE tokenizer was trained from scratch on the Greek corpus using the HuggingFace tokenizers library.

| Parameter       | Value                                     |
|-----------------|-------------------------------------------|
| Algorithm       | Byte-level BPE                            |
| Vocabulary size | 16,000                                    |
| Special tokens  | <\  endoftext\  >, <\  pad\  >            |
| Pre-tokenizer   | ByteLevel (preserves case and whitespace) |
| Decoder         | ByteLevel                                 |

**Justification:** Greek has rich morphology with many inflectional suffixes. A vocabulary of 16,000 subword tokens allows the tokenizer to capture common stems and suffixes without fragmenting too aggressively. Byte-level BPE was chosen to handle all Unicode characters natively without any unknown tokens.

## Model Architecture

A decoder-only transformer (GPT-style) was implemented in PyTorch.

| Component           | Details                                                         |
|---------------------|-----------------------------------------------------------------|
| Attention           | Causal multi-head self-attention (scaled_dot_product_attention) |
| Normalization       | Pre-LayerNorm                                                   |
| FFN                 | Linear → GELU → Linear (4× expansion)                           |
| Positional encoding | Learned embeddings                                              |
| Weight tying        | Token embedding = LM head weights                               |

## Experiment Grid

Three model sizes were each trained to 200M tokens, with evaluation checkpoints at 2M, 20M, and 200M tokens — giving 9 measurements total.

| Model   | Layers | d_model | Heads | Parameters |
|---------|--------|---------|-------|------------|
| Small   | 4      | 64      | 4     | ~1.2M      |
| Large   | 6      | 256     | 8     | ~8.9M      |
| XXLarge | 12     | 768     | 12    | ~97.5M     |

**Token checkpoints:** 2M, 20M, 200M

**Hyperparameters:** batch size 32, AdamW optimizer, lr = 3e-4, cosine annealing scheduler. Checkpointing every 50 steps with resume support.

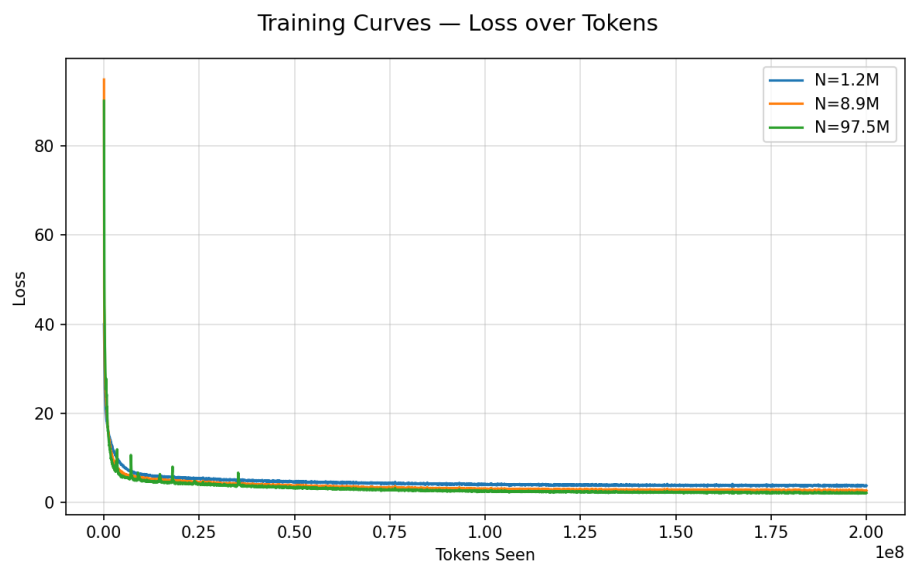
# Results

## Validation Loss Table

| N (params) | D (tokens) | FLOPs  | Val Loss    |
|------------|------------|--------|-------------|
| 1.2M       | 2M         | 1.5e13 | 13.09       |
| 1.2M       | 20M        | 1.5e14 | 5.61        |
| 1.2M       | 200M       | 1.5e15 | 3.84        |
| 8.9M       | 2M         | 1.1e14 | 11.02       |
| 8.9M       | 20M        | 1.1e15 | 4.83        |
| 8.9M       | 200M       | 1.1e16 | 2.75        |
| 97.5M      | 2M         | 1.2e15 | 9.60        |
| 97.5M      | 20M        | 1.2e16 | 4.39        |
| 97.5M      | 200M       | 1.2e17 | <b>2.23</b> |

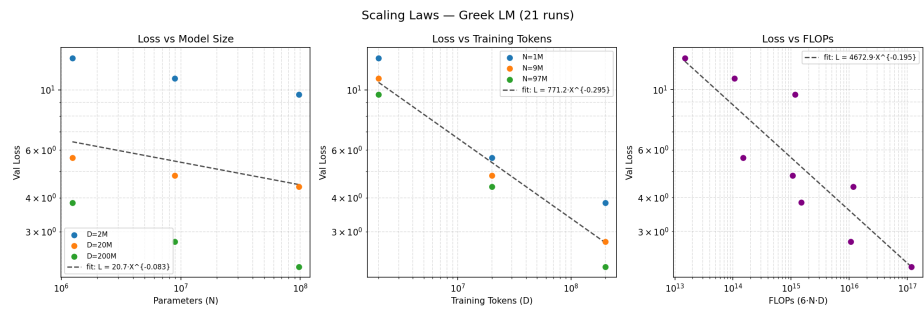
The best validation loss (2.23) was achieved by the 97.5M parameter model trained on 200M tokens ( $\sim 1.2 \times 10^{17}$  FLOPs).

## Training Curves



Training curves: loss over tokens for each model size

# Scaling Law Plots



Scaling laws: loss vs N, D, and FLOPs on log-log axes

## Scaling Law Analysis

A power law of the form  $L(X) = a \cdot X^b$  was fit to the data on log-log axes using linear regression on the log-transformed values.

### Fitted Exponents

| Axis                | a       | b (exponent)  |
|---------------------|---------|---------------|
| Parameters (N)      | 20.67   | <b>-0.083</b> |
| Training Tokens (D) | 771.19  | <b>-0.295</b> |
| FLOPs (C)           | 4672.88 | <b>-0.195</b> |

### Comparison with Literature

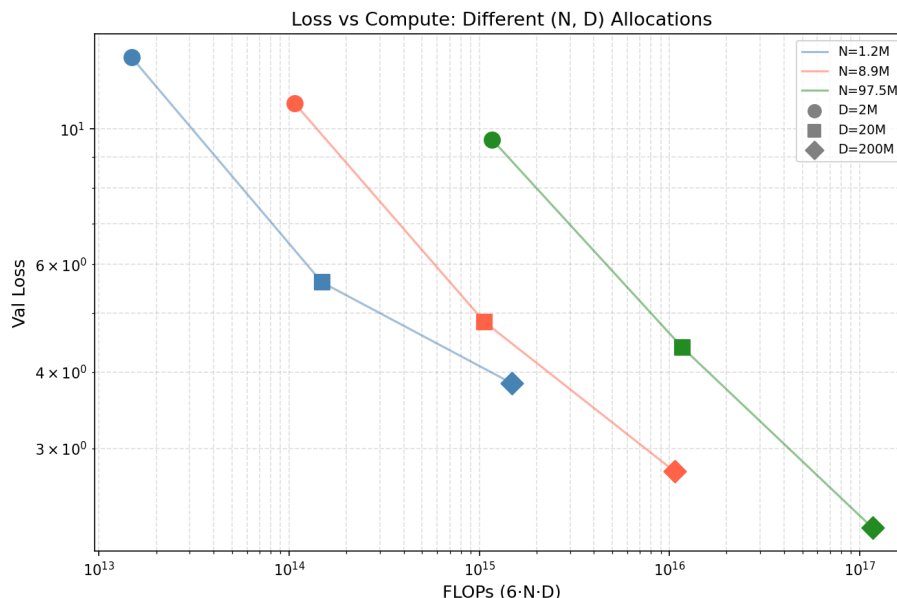
| Axis  | This work     | Kaplan et al. (2020) |
|-------|---------------|----------------------|
| N     | <b>-0.083</b> | -0.076               |
| D     | <b>-0.295</b> | -0.095               |
| FLOPs | <b>-0.195</b> | -0.050               |

The parameter exponent (-0.083) is remarkably close to the Kaplan et al. value (-0.076), suggesting that the scaling relationship with model size is robust even at small scales and with a low-resource language.

The data and FLOPs exponents are steeper than Kaplan, reflecting the small-scale regime where each additional token has a larger marginal effect. The D exponent (-0.295 vs -0.095) suggests our models are strongly data-hungry — consistent with training on a limited 2.14M-story corpus where more data consistently helps. At larger scales with more diverse data, this exponent would be expected to moderate toward the Kaplan value.

# Compute-Optimal Allocation

The following plot shows all 9 runs on a single loss-vs-FLOPs chart. Each color represents a model size and each marker shape a token count. Lines connect runs of the same model trained on increasing amounts of data.



Loss vs Compute: Different (N, D) Allocations

For a given FLOP budget, runs that use a smaller model with more data consistently achieve lower loss than larger models trained on less data. Key examples:

- The 8.9M model trained on 200M tokens ( $1.1 \times 10^{16}$  FLOPs) achieves loss 2.75, comparable to the 97.5M model trained on 20M tokens ( $1.2 \times 10^{16}$  FLOPs, loss 4.39) despite using *fewer* FLOPs. The smaller model's  $10\times$  longer training more than compensates for the  $11\times$  fewer parameters.
- At  $\sim 1 \times 10^{15}$  FLOPs: the 1.2M model on 200M tokens (loss 3.84) outperforms the 97.5M model on 2M tokens (loss 9.60) despite using similar compute.

This is consistent with the Chinchilla finding: models are typically undertrained, and allocating more tokens relative to parameters yields better performance per FLOP. The compute-optimal strategy at this scale is to train smaller models for longer.

---

## Limitations

- **Limited dataset:** The 2.14M-story corpus imposes a ceiling on training tokens — 200M token runs cycle through the dataset  $\sim 3\times$  at sequence level, which may inflate the data exponent.
- **Small scale:** All runs are far below the scale of Kaplan or Chinchilla. Exponents may not generalize beyond the 1M–100M parameter range.
- **Single seed:** Each configuration was run once. Variance across seeds is unknown.

- **Dataset quality:** The dataset is machine-translated Greek, which may introduce artifacts that affect loss measurements.
  - **No weight decay tuning:** A constant  $\text{lr}=3\text{e-}4$  and default AdamW settings were used. Per-run tuning would likely improve absolute losses.
- 

## What I Would Improve with More Compute

- **Larger grid:** More model sizes (300M+) and token counts (1B+) to test whether exponents converge toward literature values at scale.
  - **Per-run hyperparameter tuning:** Learning rate warmup and decay schedules tuned per configuration.
  - **Multiple seeds:** 3–5 seeds per run to report confidence intervals on loss and on fitted exponents.
  - **Chinchilla-style analysis:** Fitting the parametric form  $L(N,D) = A/N^\alpha + B/D^\beta + E$  jointly, rather than marginalizing each axis independently.
  - **Larger/cleaner corpus:** A native Greek corpus (not machine-translated) to reduce data artifacts.
- 

## References

- Kaplan et al. (2020) — *Scaling Laws for Neural Language Models*
- Hoffmann et al. (2022) — *Training Compute-Optimal Large Language Models (Chinchilla)*
- Sardana & Frankle (2023) — *Beyond Chinchilla-Optimal*